

```

# =====
# SISTEMA DE ANÁLISE E OTIMIZAÇÃO DE TRELIÇAS PLANAS ISOSTÁTICAS
# Compatível com Google Colab
# Autor: ChatGPT
# =====

# =====
# IMPORTAÇÕES
# =====

import math
import time
import numpy as np
import matplotlib.pyplot as plt

# =====
# CLASSE PRINCIPAL DA TRELIÇA
# =====

class TrelicaPlana:

    def __init__(self, nos, barras, cargas, apoios):

        """
        nos:
            [(x1,y1), (x2,y2), ...]

        barras:
            [(nó_i, nó_j), ...]

        cargas:
            [(Fx,Fy), ...]

        apoios:
            "fixo"    -> reage em X e Y
            "movel"   -> reage em Y
            "livre"   -> sem reação
        """

        self.nos = nos
        self.barras = barras
        self.cargas = cargas

```

```

self.apoios = apoios

self.N = len(nos)
self.B = len(barras)

self.reacoes = []
self.nom_variaveis = []

self.forcas_barras = []
self.forcas_reacoes = []

self.matriz = None
self.vetor = None
self.solucao = None

# =====
# MONTAGEM DAS INCÓGNITAS
# =====

def definir_incognitas(self):

    self.nom_variaveis = []

    # Forças nas barras
    for i in range(self.B):
        self.nom_variaveis.append(f"B{i}")

    # Reações de apoio
    for i, apoio in enumerate(self.apoios):

        if apoio == "fixo":
            self.reacoes.append((i, "Rx"))
            self.reacoes.append((i, "Ry"))

        elif apoio == "movel":
            self.reacoes.append((i, "Ry"))

    for no, tipo in self.reacoes:
        self.nom_variaveis.append(f"{tipo}_{no}")

# =====
# MONTAGEM DO SISTEMA LINEAR
# =====

```

```

def montar_sistema(self):

    self.definir_incognitas()

    num_incognitas = len(self.nom_variaveis)

    # 2 equações por nó
    A = np.zeros((2 * self.N, num_incognitas))
    b = np.zeros(2 * self.N)

    # -----
    # CONTRIBUIÇÃO DAS BARRAS
    # -----

    for idx_barra, (i, j) in enumerate(self.barras):

        xi, yi = self.nos[i]
        xj, yj = self.nos[j]

        dx = xj - xi
        dy = yj - yi

        L = math.sqrt(dx**2 + dy**2)

        c = dx / L
        s = dy / L

        # Nó i
        A[2 * i][idx_barra] += c
        A[2 * i + 1][idx_barra] += s

        # Nó j
        A[2 * j][idx_barra] -= c
        A[2 * j + 1][idx_barra] -= s

    # -----
    # REAÇÕES DE APOIO
    # -----

    col = self.B

    for no, tipo in self.reacoes:

```

```

        if tipo == "Rx":
            A[2 * no][col] = 1

        elif tipo == "Ry":
            A[2 * no + 1][col] = 1

        col += 1

# -----
# CARGAS EXTERNAS
# -----

for i, (fx, fy) in enumerate(self.cargas):

    b[2 * i] = -fx
    b[2 * i + 1] = -fy

self.matriz = A
self.vetor = b

# =====
# ELIMINAÇÃO DE GAUSS-JORDAN (MANUAL)
# =====

def gauss_jordan(self, A, b):

    n = len(b)

    # Matriz aumentada
    M = np.hstack((A.astype(float), b.reshape(-1, 1)))

    for i in range(n):

        # Pivotamento parcial
        max_row = i

        for k in range(i + 1, n):
            if abs(M[k][i]) > abs(M[max_row][i]):
                max_row = k

        M[[i, max_row]] = M[[max_row, i]]

```

```

        piv = M[i][i]

        if abs(piv) < 1e-12:
            raise ValueError("Sistema singular ou mal
condicionado.")

        # Normaliza linha pivô
        M[i] = M[i] / piv

        # Eliminação
        for j in range(n):

            if j != i:

                fator = M[j][i]
                M[j] = M[j] - fator * M[i]

    return M[:, -1]

# =====
# RESOLUÇÃO DO SISTEMA
# =====

def resolver(self):

    self.montar_sistema()

    inicio = time.time()

    self.solucao = self.gauss_jordan(
        self.matriz,
        self.vetor
    )

    fim = time.time()

    self.tempo_execucao = fim - inicio

    self.forcas_barras = self.solucao[:self.B]
    self.forcas_reacoes = self.solucao[self.B:]

# =====
# CLASSIFICAÇÃO DAS BARRAS

```

```

# =====

def classificar_barras(self):

    classificacao = []

    for f in self.forcas_barras:

        if abs(f) < 1e-6:
            classificacao.append("NULA")

        elif f > 0:
            classificacao.append("TRAÇÃO")

        else:
            classificacao.append("COMPRESSÃO")

    return classificacao

# =====
# ESTATÍSTICAS
# =====

def estatisticas(self):

    magnitudes = np.abs(self.forcas_barras)

    barra_critica = np.argmax(magnitudes)

    return {
        "forca_media": np.mean(magnitudes),
        "barra_mais_solicitada": barra_critica,
        "valor_maximo": magnitudes[barra_critica],
        "somatorio_reacoes": np.sum(np.abs(self.forcas_reacoes)),
        "tempo_execucao": self.tempo_execucao
    }

# =====
# VISUALIZAÇÃO 1 - TRELIÇA ORIGINAL
# =====

def plotar_trelica_original(self):

```

```

plt.figure(figsize=(10, 6))

# Barras
for idx, (i, j) in enumerate(self.barras):

    xi, yi = self.nos[i]
    xj, yj = self.nos[j]

    plt.plot([xi, xj], [yi, yj], 'k-', linewidth=2)

    xm = (xi + xj) / 2
    ym = (yi + yj) / 2

    plt.text(xm, ym, f"B{idx}", color='blue')

# Nós
for i, (x, y) in enumerate(self.nos):

    plt.scatter(x, y, s=80)
    plt.text(x + 0.05, y + 0.05, f"N{i}")

# Apoios
for i, apoio in enumerate(self.apoios):

    x, y = self.nos[i]

    if apoio == "fixo":
        plt.scatter(x, y - 0.1, marker='s', s=150)

    elif apoio == "movel":
        plt.scatter(x, y - 0.1, marker='^', s=150)

plt.title("Trelença Original")
plt.axis('equal')
plt.grid(True)
plt.show()

# =====
# VISUALIZAÇÃO 2 - MAPA DE CALOR
# =====

def plotar_mapa_forcas(self):

```

```

plt.figure(figsize=(10, 6))

max_force = max(abs(f) for f in self.forcas_barras)

for idx, (i, j) in enumerate(self.barras):

    xi, yi = self.nos[i]
    xj, yj = self.nos[j]

    f = self.forcas_barras[idx]

    intensidade = abs(f) / max_force

    if f >= 0:
        cor = (0, 0, intensidade) # azul
    else:
        cor = (intensidade, 0, 0) # vermelho

    plt.plot(
        [xi, xj],
        [yi, yj],
        linewidth=4,
        color=cor
    )

for i, (x, y) in enumerate(self.nos):

    plt.scatter(x, y, color='black')
    plt.text(x + 0.03, y + 0.03, f"N{i}")

plt.title("Mapa de Forças Internas")
plt.axis('equal')
plt.grid(True)
plt.show()

# =====
# VISUALIZAÇÃO 3 - GRÁFICO DE BARRAS
# =====

def plotar_grafico_barras(self):

    plt.figure(figsize=(12, 5))

```



```

indices = np.arange(self.B)

plt.bar(indices, self.forcas_barras)

plt.xlabel("Barra")
plt.ylabel("Força Interna (N)")
plt.title("Magnitudo das Forças nas Barras")

plt.grid(True)
plt.show()

# =====
# VISUALIZAÇÃO 4 - HISTOGRAMA
# =====

def plotar_histograma(self):

    plt.figure(figsize=(10, 5))

    plt.hist(self.forcas_barras, bins=10)

    plt.xlabel("Força")
    plt.ylabel("Frequência")

    plt.title("Distribuição das Forças Internas")

    plt.grid(True)
    plt.show()

# =====
# RELATÓRIO
# =====

def relatorio(self):

    classificacao = self.classificar_barras()
    stats = self.estatisticas()

    print("\n" + "=" * 60)
    print("RESULTADOS DA TRELIÇA")
    print("=" * 60)

    for i, f in enumerate(self.forcas_barras):

```

```

        print(
            f"Barra {i:02d} | "
            f"Força = {f:10.2f} N | "
            f"{classificacao[i]}"
        )

    print("\nREAÇÕES DE APOIO")

    for i, r in enumerate(self.forcas_reacoes):

        print(f"{self.nom_variaveis[self.B + i]} = {r:.2f} N")

    print("\nESTATÍSTICAS")

    print(f"Força média: {stats['forca_media']:.2f} N")
    print(f"Barra mais solicitada:
{stats['barra_mais_solicitada']}")
    print(f"Força máxima: {stats['valor_maximo']:.2f} N")
    print(f"Somatório das reações: {stats['somatorio_reacoes']:.2f}
N")

    print(f"Tempo de execução: {stats['tempo_execucao']:.6f} s")

# =====
# GERADOR AUTOMÁTICO DE TRELIÇA WARREN
# =====

def gerar_trelica_warren(n_paineis=4, carga=-1000):

    """
    Gera automaticamente uma treliça Warren.

    Quanto maior n_paineis:
        maior N e B
    """

    nos = []

    # Nós inferiores
    for i in range(n_paineis + 1):
        nos.append((i * 2, 0))

```

```

# Nós superiores
for i in range(n_paineis):
    nos.append((i * 2 + 1, 1.5))

barras = []

# Barras inferiores
for i in range(n_paineis):
    barras.append((i, i + 1))

# Barras diagonais
for i in range(n_paineis):

    barras.append((i, n_paineis + 1 + i))
    barras.append((i + 1, n_paineis + 1 + i))

# Barras superiores
for i in range(n_paineis - 1):
    barras.append(
        (
            n_paineis + 1 + i,
            n_paineis + 2 + i
        )
    )

N = len(nos)

cargas = [(0, 0) for _ in range(N)]

# carga central
cargas[n_paineis // 2] = (0, carga)

apoios = ["livre"] * N

apoios[0] = "fixo"
apoios[n_paineis] = "movel"

return nos, barras, cargas, apoios

# =====
# EXECUÇÃO PRINCIPAL
# =====

```

```
# -----
# TRELIÇA TESTE PADRÃO
# -----

# N = 4 nós aproximadamente
nos, barras, cargas, apoios = gerar_trelica_warren(
    n_paineis=2,
    carga=-1000
)

# -----
# CRIA OBJETO
# -----

trelica = TrelicaPlana(
    nos,
    barras,
    cargas,
    apoios
)

# -----
# RESOLVE
# -----

trelica.resolver()

# -----
# RELATÓRIO
# -----

trelica.relatorio()

# -----
# GRÁFICOS OBRIGATÓRIOS
# -----

trelica.plotar_trelica_original()

trelica.plotar_mapa_forcas()

trelica.plotar_grafico_barras()
```

```

trelica.plotar_histograma()

# =====
# TESTES DE ESCALABILIDADE
# =====

print("\n" + "=" * 60)
print("TESTE DE ROBUSTEZ")
print("=" * 60)

for paineis in [2, 4, 8, 16]:

    nos, barras, cargas, apoios = gerar_trelica_warren(
        n_paineis=paineis,
        carga=-1000
    )

    modelo = TrelicaPlana(
        nos,
        barras,
        cargas,
        apoios
    )

    modelo.resolver()

    print(
        f"Painéis: {paineis:2d} | "
        f"Nós: {modelo.N:2d} | "
        f"Barras: {modelo.B:2d} | "
        f"Tempo: {modelo.tempo_execucao:.6f} s"
    )

```

=====

RESULTADOS DA TRELIÇA

=====

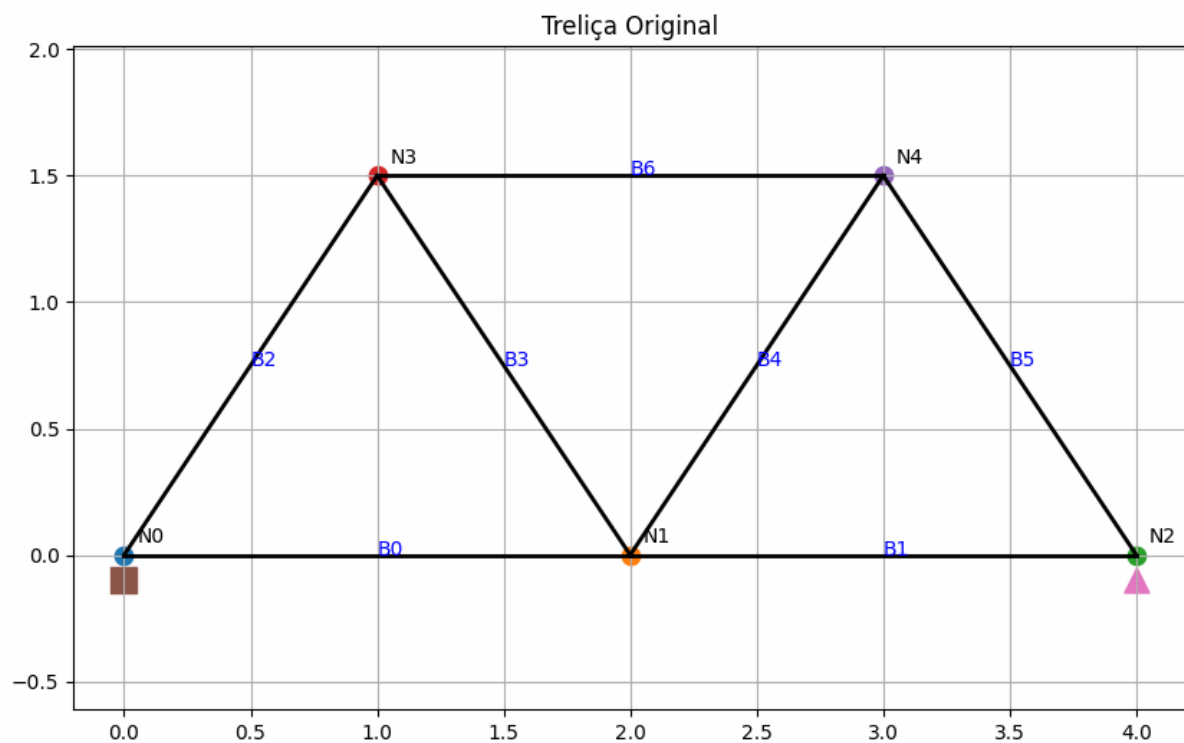
Barra 00 | Força = 333.33 N | TRAÇÃO
Barra 01 | Força = 333.33 N | TRAÇÃO
Barra 02 | Força = -600.93 N | COMPRESSÃO
Barra 03 | Força = 600.93 N | TRAÇÃO
Barra 04 | Força = 600.93 N | TRAÇÃO
Barra 05 | Força = -600.93 N | COMPRESSÃO
Barra 06 | Força = -666.67 N | COMPRESSÃO

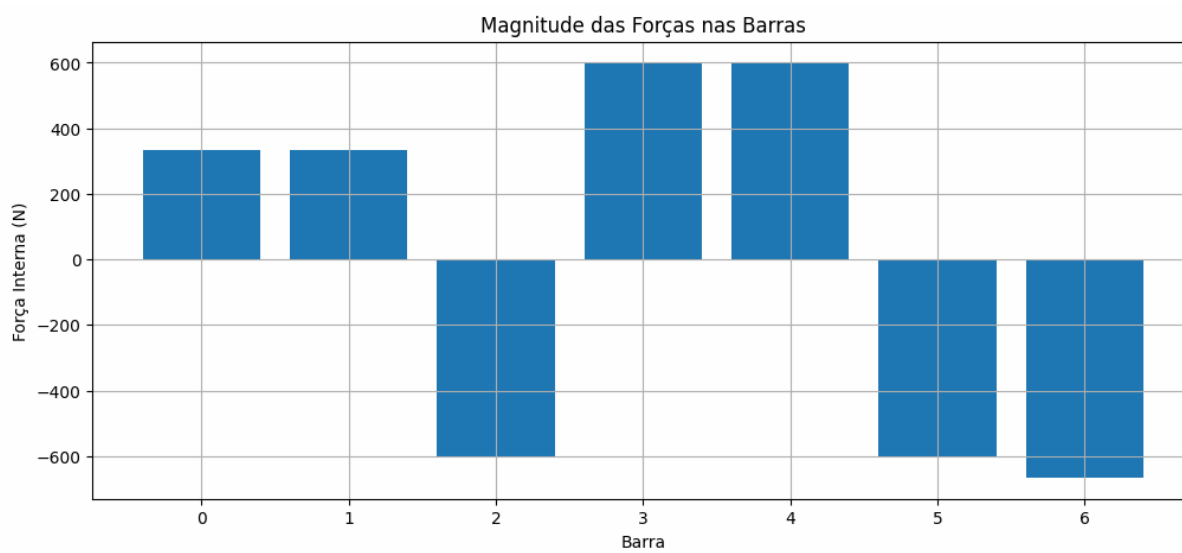
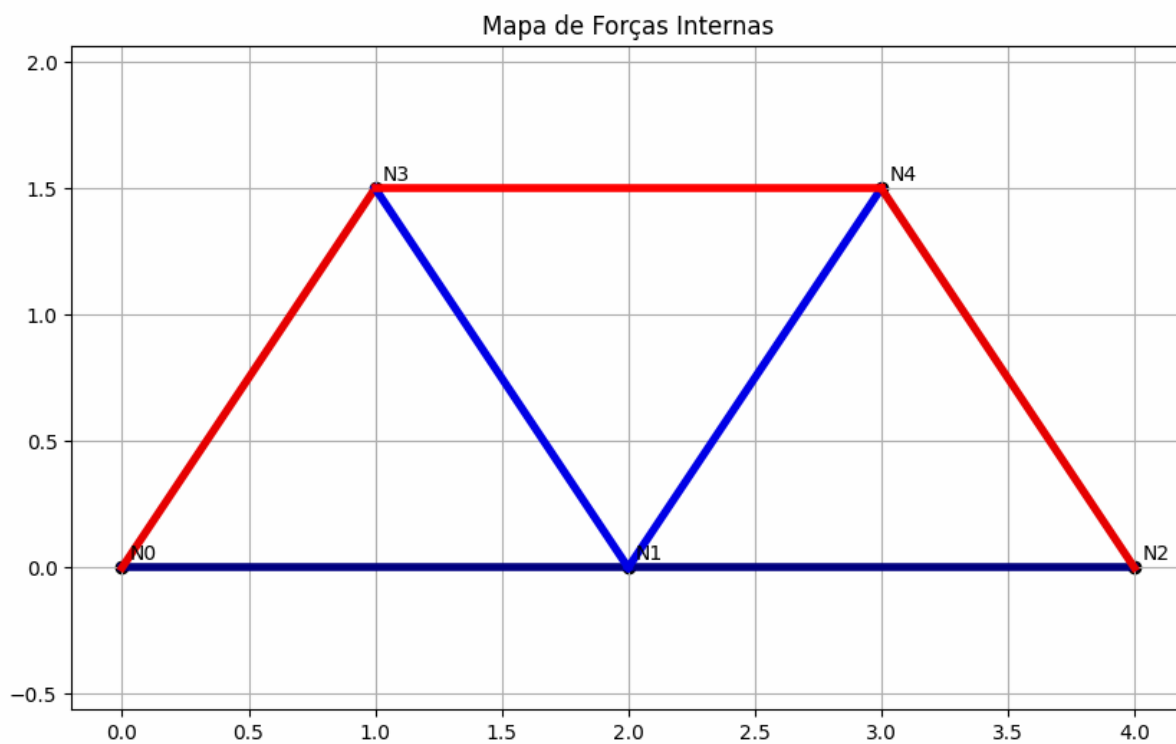
REAÇÕES DE APOIO

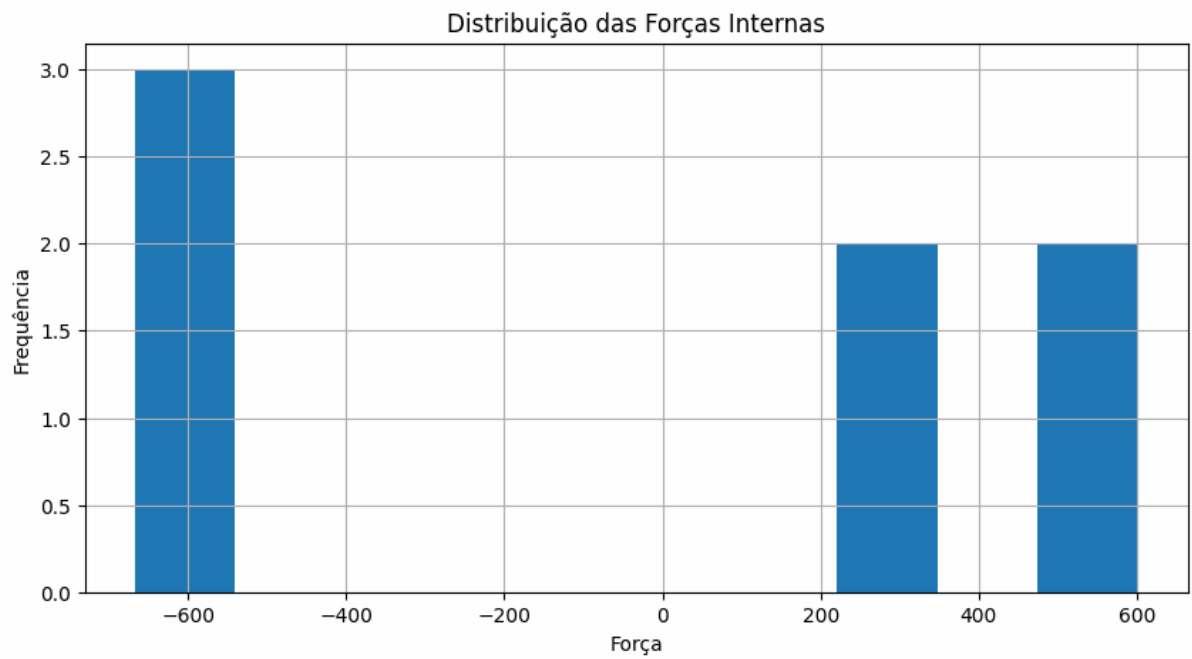
R_{x_0} = 0.00 N
R_{y_0} = 500.00 N
R_{y_2} = 500.00 N

ESTATÍSTICAS

Força média: 533.86 N
Barra mais solicitada: 6
Força máxima: 666.67 N
Somatório das reações: 1000.00 N
Tempo de execução: 0.000509 s







=====

TESTE DE ROBUSTEZ

=====

Painéis: 2 | Nós: 5 | Barras: 7 | Tempo: 0.002366 s
Painéis: 4 | Nós: 9 | Barras: 15 | Tempo: 0.002107 s
Painéis: 8 | Nós: 17 | Barras: 31 | Tempo: 0.007427 s
Painéis: 16 | Nós: 33 | Barras: 63 | Tempo: 0.027889 s